

Evolutionary versus Random Search for Feed-Forward Network Topology: A Small, Fully Reproducible Study

Vasili Gavrilov

2026

Abstract

We ask a plain question: does a genetic algorithm that evolves the topology of a feed-forward neural network beat random search, given the same compute? We answer it with a small, dependency-free engine written in C, in which every candidate network is trained by backpropagation, every run is reproducible from a seed, and every experiment is one shell command. On a synthetic classification task where the network topology genuinely affects generalization, we report three findings. First, a *self-adaptive* mutation rate—one that is carried in the genome and evolves with it—beats a fixed rate by a small but consistent margin in a controlled comparison. Second, once the search space is enlarged, the genetic algorithm no longer beats random search at any compute budget we tried (96 to 768 candidate evaluations); it hovers near parity. Third, this is what the exploration/exploitation account predicts: at a fixed budget the genetic algorithm commits to the best of a few global samples and then refines locally, while random search keeps sampling the whole space. The results reproduce, at miniature scale, the known strength of random search as a baseline. We give the exact settings, population sizes, and per-run statistics so the study can be repeated; we also state plainly where our budget experiments are not perfectly controlled, and specify the clean experiment that would settle the open question.

1 Introduction

Neural architecture search (NAS) automates a choice a human normally makes: the shape of the network—how many layers, how wide, how connected. The idea is old; the name is recent. Modern NAS methods differ mainly in their *search strategy*: reinforcement learning [5], gradient-based relaxation [8], Bayesian optimization, random search, and evolutionary search [10]. The evolutionary branch is the subject here. It comes in two flavors that are easy to confuse: classical *neuroevolution* such as NEAT [3] evolves the weights *and* the topology together, using no gradient; *evolutionary NAS* [6, 7] evolves the topology but trains the weights by gradient descent. Our system is the second kind: it evolves the topology and trains each candidate’s weights by backpropagation.

A recurring, humbling result in this field is that *random search is hard to beat*. Bergstra and Bengio [4] showed plain random search is a strong baseline for hyper-parameter optimization; Li and Talwalkar [9] showed random search (with weight sharing) is competitive with leading NAS methods, and documented a reproducibility problem in the field along the way. Our contribution is not a new method. It is a small, honest, fully reproducible test of the basic question—*does evolution beat random search here?*—in a system simple enough that every number in this paper can be regenerated from source with one command. The implementation and these experiments are open [11].

2 Method

2.1 Network and training

A network is a plain feed-forward stack. For layer ℓ with weight matrix $W^{(\ell)}$ and bias $b^{(\ell)}$, the forward pass is

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = g(z^{(\ell)}), \quad (1)$$

with g the logistic sigmoid $g(z) = 1/(1 + e^{-z})$. Weights are trained by backpropagation (the generalized delta rule) with momentum: with local error signal $\beta_i = (d_i - a_i)g'(z_i)$ at output units and $\beta_i = g'(z_i) \sum_k W_{ki}^{(\ell+1)}\beta_k^{(\ell+1)}$ at hidden units, the update is

$$\Delta W^{(\ell)}(t) = r \beta^{(\ell)}(a^{(\ell-1)})^\top + \alpha \Delta W^{(\ell)}(t-1), \quad (2)$$

with learning rate r and momentum α . Weights are initialized uniformly in $[-2.4/f_i, 2.4/f_i]$, where f_i is the fan-in. Training is online (one example at a time), so a worker’s memory footprint is bounded regardless of dataset size.

A *worker* is one process. Given a topology, a dataset, and a seed, it builds the network, trains it for a fixed number of epochs on the training split, and prints one line: the topology and its *fitness*, the mean squared error on the held-out test split. Lower is better. All arithmetic uses 32-bit floats; all randomness flows through a seeded 32-bit xorshift generator, so the same seed gives the same result on any machine.

2.2 Genome, mutation, and self-adaptation

A *genome* is a topology: the list of layer widths $(d_0, \dots, d_L)$, with the input width d_0 and output width d_L fixed by the problem and the hidden layers free. One *mutation* makes a single small, local change: perturb one hidden width by ± 1 , insert one hidden layer, or delete one hidden layer, staying within bounds on the number of hidden layers ($L_{\max}$) and their width ($W_{\max}$). There is no crossover: recombining two arbitrary topologies is ill-posed (the competing-conventions problem), and mutation-only evolution is the current default in evolutionary NAS [7].

The mutation rate is itself a hyper-parameter, and a poorly chosen rate can make a genetic algorithm worse than random. Rather than fix it by hand, we let it evolve: each genome carries a mutation rate σ , and reproduction is *self-adaptive* in the sense of evolution strategies [2]. An offspring inherits its parent’s rate, perturbs it log-normally, and then applies that many mutations:

$$\sigma' = \sigma \cdot \exp(\tau \mathcal{N}(0, 1)), \quad \text{number of mutations} = \text{round}(\sigma'), \quad (3)$$

clamped to $[\sigma_{\min}, \sigma_{\max}]$, with $\tau = 0.35$. Selection then favors not just good topologies but the rates that produced them, so the rate tunes itself: it tends to rise early (to explore) and fall later (to refine).

2.3 Evolutionary search and the random control

The search keeps a population of P genomes for G generations, retaining the best k (elitism) and refilling the rest by reproducing from the elite (Algorithm 1). The *matched random-search control* draws P fresh random genomes every generation and keeps its own best-so-far; over G generations it therefore makes exactly the same number of evaluations, $P \times G$, as the genetic algorithm. Every candidate is evaluated by its own worker process; the processes are launched in parallel, one per CPU, and hand back plain text. Crucially, the genetic algorithm’s *initial* population is drawn from the same random distribution as the control, so both start from the same place; they differ only in what they do next.

Algorithm 1 Evolutionary search (one run), with matched random control

```
1: population  $\leftarrow P$  random genomes; control-best  $\leftarrow \infty$ 
2: for generation = 1 to  $G$  do
3:   evaluate the population in parallel (one worker process per genome)
4:   keep the best  $k$  (elite); update the genetic best-so-far
5:   refill: each new genome reproduces from a random elite (self-adaptive or fixed rate)
6:   control: evaluate  $P$  fresh random genomes; update control-best
7: end for
8: return genetic best-so-far, control best-so-far
```

Table 1: Self-adaptive versus fixed mutation, small space, 10 seeds. Entries are genetic wins out of 10 (mean gap). Self-adaptation wins more often and by a wider margin at every starting rate.

starting rate ($-M$)	fixed	self-adaptive
1	5 (+0.0021)	8 (+0.0026)
3	5 (+0.0004)	6 (+0.0017)
6	5 (+0.0009)	7 (+0.0023)

2.4 The task

To ask whether topology matters, the task must be one where it does—and which the networks can actually learn, so that better topologies score better. We generate a synthetic classification task, reproducibly, from a seed. Inputs x are uniform in $[-1, 1]^d$; the label is

$$y = [\sin(f w \cdot x) > 0], \quad (4)$$

for a fixed random projection w and frequency f , with a fraction of labels flipped as noise. Higher f makes more decision stripes and demands more capacity. We screened for learnability: at the settings below, trained networks reach a test MSE around 0.05–0.08, well under the 0.25 that predicting a constant 0.5 would score, so topology genuinely drives generalization. (We discarded one early run at $d = 8, f = 4$ precisely because *no* topology learned it in the training budget—test MSE ≈ 0.27 , worse than chance—which leaves nothing for either search to distinguish.)

2.5 Protocol and metrics

Each *run* uses a fresh task (task seed = run index) and its own search seed, so results are averaged across task instances, not a single lucky one. Within a run the genetic algorithm and the control see the same task and the same 80/20 train/test split. We report two metrics over the runs: the *win rate*, the fraction of runs in which the genetic best beat the control best, and the *mean gap*, the average of (control MSE – genetic MSE), positive when the genetic algorithm is better.

3 Experiments

3.1 Self-adaptive versus fixed mutation (controlled)

We first compare a fixed mutation rate against the same rate left free to self-adapt, in a small search space ($L_{\max} = 3, W_{\max} = 16$), holding everything else equal ($d = 4$, 600 samples, $f = 3$; $P = 10, G = 8$, budget 80; 1200 training epochs; 10 seeds). This is a clean A/B: only the adaptation is toggled.

Table 2: Bigger space ($\leq 5 \times 28$), fixed budget 96, 15 seeds. The genetic algorithm does worse than random.

mode	genetic wins / 15	mean gap
fixed	4	−0.0029
self-adaptive	5	−0.0026

Table 3: Search budget in the bigger space. **This is not a controlled sweep:** to bound wall-clock, the per-candidate training (epochs, samples) and the elitism were also changed as the budget grew, so differences across rows conflate budget with training and selection. Read within a row, not across.

budget ($P \times G$)	samples/epochs/elite	seeds	fixed wins (gap)	self-adaptive wins (gap)
96	1000/1000/2	15	4 (−0.0029)	5 (−0.0026)
384	800/700/2	8	3 (+0.0008)	4 (−0.0006)
768	600/500/8	8	5 (+0.0021)	3 (−0.0006)

Table 1 shows the effect. A fixed rate is about a coin flip against random (5 of 10) whatever the rate; self-adaptation lifts this to 6–8 of 10 and widens the gap, at every starting rate. The benefit is real but modest. (An earlier 3-seed pass had shown a fixed rate *losing* at one rate and a mutation-rate “sweet spot”; ten seeds regressed both to the mean. This is why the counts below use as many seeds as the compute allowed.)

3.2 A bigger search space, fixed budget

Enlarging the space to $L_{\max} = 5$, $W_{\max} = 28$ while holding the budget at $P \times G = 96$ (and $d = 4$, 1000 samples, $f = 3$, 1000 epochs, 15 seeds) reverses the picture (Table 2): the genetic algorithm now does *worse* than random, winning 4–5 of 15 with a negative mean gap.

The reason is budget, not the search. Both make 96 evaluations, but the genetic algorithm spends only the first generation (P genomes) sampling the space *globally*; every later generation refines *locally* around the elite. Random search spends all 96 evaluations globally. In a larger space, random’s broader coverage beats committing to the best of P global samples and polishing it, because local mutation cannot cross to the better regions random happens to find. Elitism guarantees the genetic algorithm never *loses* its best, but that best came from too few global draws.

3.3 Scaling the budget (uncontrolled)

If the deficit is budget, more budget should help. Table 3 scales the search budget from 96 to 768 evaluations in the same bigger space. The gap does narrow from about −0.003 toward zero, and at 768 the fixed variant edges positive (5/8, +0.0021)—but the genetic algorithm never decisively overtakes random, and the win counts (on 8 seeds) are noisy and non-monotone.

We flag the confound rather than hide it: as the budget rose we cut training epochs to keep each run near forty minutes, and fewer epochs flatten the fitness signal, which pushes the gap toward zero on its own. So Table 3 is *suggestive*, not conclusive. The controlled experiment—vary only G , hold task, epochs, samples, and elitism fixed—is specified in the conclusion and is the clean next step.

4 Discussion

Three things hold up. **(1)** Random search is a strong baseline: across every bigger-space setting we tried, the genetic algorithm did not reliably beat it, hovering near parity. This reproduces, from scratch, the lesson of [4, 9]. **(2)** Self-adaptive mutation beats a fixed rate, reliably and by a little, in the one comparison we controlled cleanly (Table 1); its advantage did not clearly persist in the smaller, noisier bigger-space runs, so we do not claim it is universal. **(3)** The exploration/exploitation trade-off explains the pattern: directed local search (the genetic algorithm) beats broad sampling (random) only when the space is small enough for its limited global budget to cover, or when the budget scales with the space; otherwise broad sampling wins.

Limitations. The scale is small and the task is synthetic; the networks are dense feed-forward only; 8–15 seeds is coarse for win-count comparisons; the fitness landscape over topologies is fairly flat, which is unfavorable to any directed search; and, as noted, our budget experiments are not controlled. We used mean squared error as fitness, a proxy for classification error. None of this overturns the three findings, but each bounds how far to read them.

5 Conclusion and future work

The twenty-five-year-old idea—let the network find its own shape—runs, and it produces an honest answer: at this scale, evolving topologies does not beat random search, and making the search space bigger without making the budget bigger makes matters worse. The clear next experiments are three. First, a *controlled* budget sweep: hold the task, training, and selection fixed and vary only the number of generations, to isolate the effect of budget. Second, a *structured* search space (skip connections, per-layer choices) in which local moves have real gradient to climb, the setting where a directed search should finally pay. Third, a bridge to a standard tabular NAS benchmark (NAS-Bench-101/201), which supplies pre-computed accuracies over a fixed search space and would let the same comparison run over thousands of seeds cheaply. Each is one command away in the released code.

Reproducibility

Every result above is regenerated from [11] with a single command. The engine builds with `make` (a C99 compiler, no dependencies) into three programs: `smbpann` (the worker), `evolve` (the search and control), and `gentask` (the task generator); `scripts/benchmark.sh` runs a whole experiment and prints the win rate and mean gap. The controlling variables are environment variables: `DIM`, `N`, `FREQ`, `NOISE` (the task); `LMAX`, `WMAX` (the search space); `POP`, `GENS`, `ELITE`, `MUT`, `ADAPT` (the search); and `RUNS`, `EPOCHS`. For example, Table 2 is

```
DIM=4 N=1000 FREQ=3 LMAX=5 WMAX=28 EPOCHS=1000 POP=12 GENS=8 RUNS=15
MUT=2 ADAPT=1 scripts/benchmark.sh
```

and `ADAPT=0` gives the fixed-rate row. Each run uses a fresh task seed and a fixed 32-bit xorshift generator, so the numbers are identical on re-running.

References

- [1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.
- [2] I. Rechenberg. *Evolutionsstrategie*. 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981 (self-adaptive mutation in evolution strategies).

- [3] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [4] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [5] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [6] E. Real et al. Large-Scale Evolution of Image Classifiers. *ICML* 2017.
- [7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [8] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [9] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.
- [10] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [11] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, <https://github.com/Anode1/SMBPANN>.